

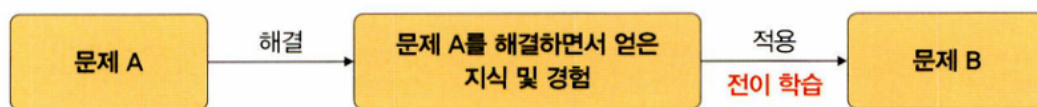
Chapter 5. 합성곱 신경망 - Part2

5.3 전이 학습

아주 큰 데이터셋(ex. 이미지넷)을 써서 훈련된 모델의 가중치를 가져와 과제에 맞게 보정해서 사용하는 것

- 사전 훈련된 모델(네트워크): 아주 큰 데이터셋을 사용하여 훈련된 모델

▼ 그림 5-29 전이 학습



- 전이학습 방법: 특성 추출, 미세 조정 기법

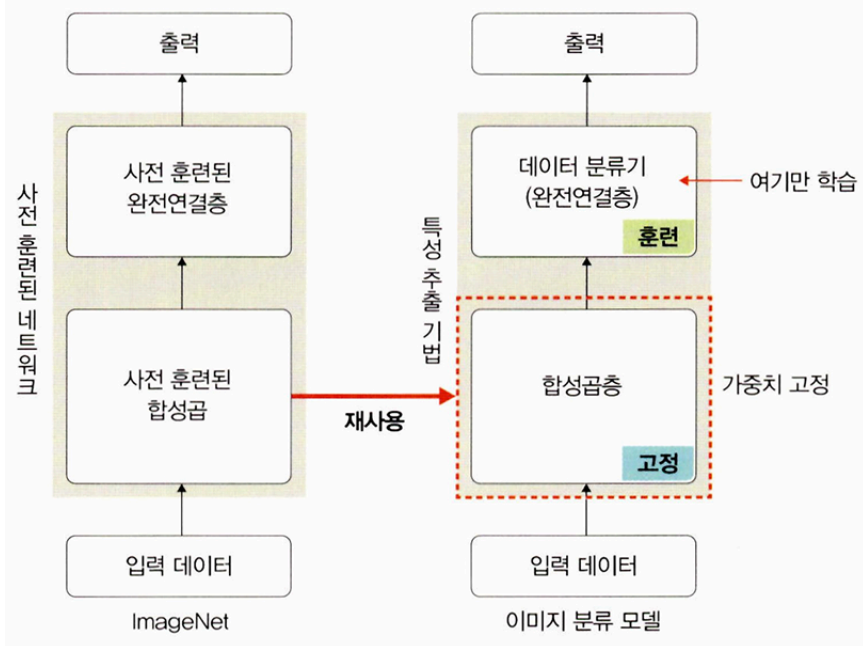
5.3.1 특성 추출 기법

ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 후 마지막에 완전연결층 부분만 새로 만들

⇒ 학습할 때는 마지막 완전연결층만 학습(나머지 계층들은 학습 X)

- 특성 추출 구성
 - 합성곱층: 합성곱층과 풀링층으로 구성
 - 데이터 분류기(완전연결층): 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스에 분류하는 부분
- 사전 훈련된 네트워크의 합성곱층(가중치 고정)에 새로운 데이터를 통과, 출력을 데이터 분류기에서 훈련 시킴
- 사용 가능한 이미지 분류 모델
 - Xception, Inception V3, ResNet50, VGG16, VGG19, MobileNet
- 특성 추출 기법 도식화

♥ 그림 5-30 특성 추출 기법



- 특성 추출에 필요한 라이브러리

```
pip install opencv-python
```

#코드 5-12. 라이브러리 호출

```
import os
import time
import copy
import glob
import cv2 # 앞에서 설치한 OpenCV 라이브러리
import shutil

import torch
import torchvision # 컴퓨터 비전 용도의 패키지
import torchvision.transforms as transforms # 데이터 전처리를 위해 사용되는
패키지
import torchvision.models as models # 다양한 파이토치 네트워크를 사용할 수
있도록 도와주는 패키지
import torch.nn as nn
```

```
import torch.optim as optim
from torch.utils.data import DataLoader

import matplotlib.pyplot as plt
```

#코드 5-13. 이미지 데이터 전처리 방법 정의

```
data_path='/content/drive/MyDrive/Euron9_DL/data' #이미지 데이터가 위치한
경로 지정
```

```
#torchvision.transform: 이미지 데이터를 변환하여 모델의 입력으로 사용하게 변환
transform = transforms.Compose(
    [
        transforms.Resize([256,256]), #이미지 크기 조정. 256 x 256 크기로 이미
지 데이터 조정
        transforms.RandomResizedCrop(224), #이미지를 랜덤한 크기 및 비율로 자
름
        transforms.RandomHorizontalFlip(), #이미지를 랜덤하게 수평으로 뒤집는
기능
        transforms.ToTensor()] #이미지 데이터를 텐서로 변환
    )
```

Resize Vs. RandomResizedCrop



Resize : 합성곱층을 통과하기 위해 이미지 크기를 조정하는 전처리 과정

RandomResizedCrop: 데이터 확장 용도, 이미지를 랜덤한 비율로 자른 후 데
이터 크기 조정

```
#datasets.ImageFolder: 데이터로더가 데이터를 불러올 대상(/경로)과 방법을 정의
train_dataset = torchvision.datasets.ImageFolder(
    data_path, #불러올 데이터가 위치한 경로
    transform=transform ) #이미지 데이터에 대한 전처리
```

```
#dataloader: 데이터를 불러올 부분으로 앞에서 정의한 ImageFolder을 데이터로더
에 할당
train_loader = torch.utils.data.DataLoader(
    train_dataset,#데이터셋 지정
    batch_size=32, #한 번에 불러올 데이터양을 결정하는 배치 크기 설정
    num_workers=8, #데이터를 불러올 하위 프로세스를 몇 개 사용할지 설정(너무 많
으면 오류/메모리 부족 현상)
    shuffle=True ) #데이터를 무작위로 섞을지 설정
```

```
print(len(train_dataset)) #train_dataset에 포함된 데이터 개수 출력
```

- 전체 이미지 사용 시, CPU 과부하 발생 → 일부 이미지만 사용
 - mxnet 패키지 설치
 - 데이터 확장 코드 적용
 - RandomResizedCrop을 적용하기 위한 함수 생성

```
#불러온 이미지에 RandomResizedCrop을 적용하기 위한 함수 생성
def show_images(imgs, num_rows, num_cols, scale=2):
    aspect_ratio = imgs[0].shape[0]/imgs[0].shape[1] #확장할 이미지의
    크기 조정
    figsize = (num_cols * scale, num_rows*scale*aspect_ratio)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    for i in range(num_rows):
        for j in range(num_cols):
            axes[i][j].imshow(imgs[i*num_cols+j].asnumpy())
            axes[i][j].axes.get_xaxis().set_visible(False) #x축 전체를 숨김
            axes[i][j].axes.get_yaxis().set_visible(False) #y축 전체를 숨김
    plt.subplots_adjust(hspace=0.1, wspace=0)
    return axes

def apply(img, aug, num_rows=2, num_cols=4, scale=3):
    Y=[aug(img) for _ in range(num_rows*num_cols)] #다양한 샘플을 얻
    기 위해 여러 번 데이터 확장 적용
    show_images(Y, num_rows, num_cols, scale)
```

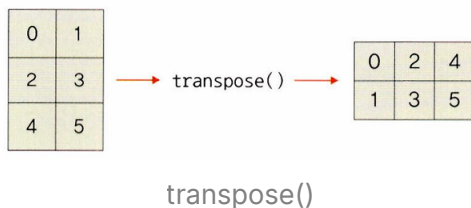
```
#RandomResizedCrop이 적용된 이미지 출력
shape_aug = transforms.RandomResizedCrop(size=(200,200), #출력 크기 200X200으로 조정
                                          scale=(0.1, 1), #면적 비율 0.1~1(10~100%) 범위 내에서 무작위로 자름
                                          ratio=(0.5, 2)) #면적의 너비와 높이 비율을 0.5~2 범위 내에서 무작위 조절
apply(example_image, shape_aug)
```

- ratio를 통해 너비와 비율 조절 후, scale을 통해 자르고, 최종적으로 출력할 size로 조정

#코드 5-14. 학습에 사용될 이미지 출력

```
samples, labels = next(iter(train_loader)) #①
classes = {0: 'cat', 1: 'dog'} #개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24): #24개의 이미지 데이터 출력
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) #레이블 정보(클래스)를 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0))) #②
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

- ① 반복자를 사용하기 위해서 iter(), next() 사용
- ② np.transpose : 행과 열을 바꿔, 행렬의 차원을 바꿈
 - 행렬의 차원 변경 이유: 행렬의 내적 연산
 - 행과 열의 수가 달라 내적이 불가능할 때, 차원 조절 필요



곱셈, 덧셈

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

A B C

$$\begin{bmatrix} c_{11} \\ c_{21} \\ c_{31} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} \end{bmatrix}$$

행렬의 내적

#코드 5-15. 사전 훈련된 모델 내려받기

```
resnet18 = models.resnet18(pretrained=True)
```

ResNet18



50개의 계층으로 구성된 합성곱 신경망으로, ImageNet 데이터베이스의 100만 개가 넘는 영상을 이용하여 훈련된 신경망으로 전이 학습에 사용되도록 사전 훈련된 모델을 제공한다.

- 단점: 입력 제약이 매우 크고, 충분한 메모리가 없으면 학습 속도가 느리다

사전 훈련된 모델

#파이토치는 다음과 같은 방법으로 무작위의 가중치로 모델을 구성

```
import torchvision.models as models
resnet18 = models.resnet18()
alexnet = models.alexnet()
vgg16 = models.vgg16()
squeezenet = models.squeezenet1_0()
densenet = models.densenet161()
inception = models.inception_v3()
googlenet = models.googlenet()
shufflenet = models.shufflenet_v2_x1_0()
mobilenet_v2 = models.mobilenet_v2()
mobilenet_v3_large = models.mobilenet_v3_large()
mobilenet_v3_small = models.mobilenet_v3_small()
resnext50_32x4d = models.resnext50_32x4d()
wide_resnet50_2 = models.wide_resnet50_2()
mnasnet = models.mnasnet1_0()
```



#사전 학습된 모델을 사용할 수도 있음 - pretrained=True로 설정하여 사용

#코드 5-16. 사전 훈련된 모델의 파라미터 학습 유무 지정

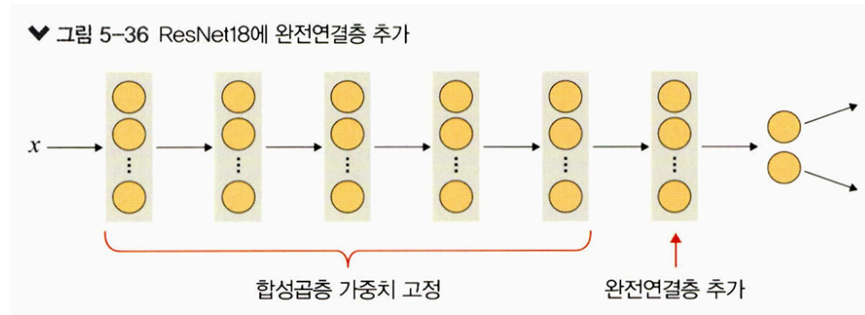
```
def set_parameter_requires_grad(model, feature_extracting=True):
    if feature_extracting:
        for param in model.parameters():
            param.requires_grad=False #①
set_parameter_requires_grad(resnet18)
```

- ① 역전파 중 파라미터들에 대한 변화 계산 필요 X
모델의 일부(합성곱층, 풀링층)를 고정, 학습할 때는 param.requires_grad=False로 설정

#코드 5-17. ResNet18에 완전연결층 추가

```
resnet18.fc = nn.Linear(512, 2) #2는 클래스가 두 개라는 의미
```

- 내려받은 ResNet18의 마지막 부분에 완전연결층 추가 → 개, 고양이 클래스 분류 용도



#코드 5-18. 모델의 파라미터 값 확인

- 파라미터는 weight와 bias가 사용

#코드 5-19. 모델 객체 생성 및 손실 함수 정의

- 모델의 합성곱층 가중치는 고정, 완전연결층은 학습
- 모델의 마지막에 완전연결층을 추가한 모델을 보여줌

#코드 5-20. 모델 학습을 위한 함수 생성

- 모델 재사용을 위해 저장
- 모델의 정확도와 오차를 반환

#코드 5-21. 파라미터 학습 결과를 옵티마이저에 전달

- weight, bias값들이 업데이트되고 옵티마이저에 전달

```
fc.weight  
fc.bias
```


#코드 5-22. 모델 학습

- 모델 훈련을 위해 전달되는 파라미터는 모델, 학습 데이터, 손실 함수, 옵티마이저, 장치
- 손실 함수는 CrossEntropyLoss로 지정
- 모델 학습 결과

```
Epoch 12/12
-----
Loss: 0.1788 Acc: 0.9247

Training complete in 10m 11s
Best Acc: 0.940260
```

약 94%로 상당히 높은 정확도를 보여, 훈련 데이터의 학습이 잘 되었음

#코드 5-23. 테스트 데이터 호출 및 전처리

- 학습 데이터 때와 동일한 방식으로 전처리
- 테스트 데이터는 98개

#코드 5-24. 테스트 데이터 평가 함수 생성

- glob: 현재 디렉터리에서 원하는 파일들만 추출하여 가져올 때 사용

```
saved_models = glob.glob("../Euron9_DL/data/catanddog/"+"*.pth")
```

경로에서 pth 확장자(훈련 데이터로 모델을 훈련시킬 때 생성된 파일)를 갖는 파일 가져 오기

- torch.max: 주어진 텐서 배열의 최댓값이 들어 있는 index 반환 함수

```
_, preds = torch.max(outputs.data, 1)
```

- preds.eq(labels): preds 배열과 labels가 일치하는지 검사하는 용도로 사용

```
running_corrects += preds.eq(labels.cpu()).int().sum()
```

뒤의 .sum()은 모델의 예측 결과와 정답이 일치하는 것들의 개수 합을 숫자로 출력

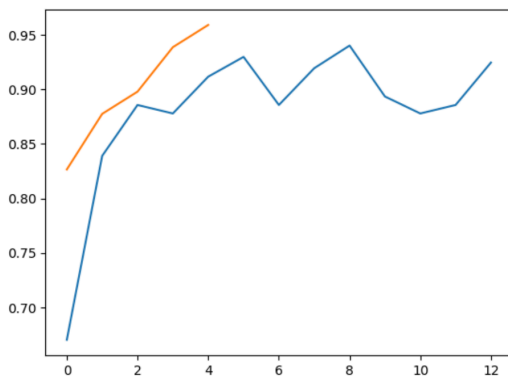
#코드 5-25. 테스트 데이터를 평가 함수에 적용

```
Validation complete in 2m 28s  
Best Acc: 0.959184
```

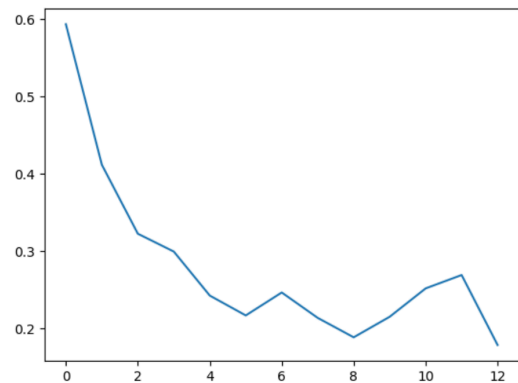
- 약 95% 정도의 높은 정확도
- 사전 훈련된 모델 사용 시, 손쉽게 모델 학습 가능

#코드 5-26, 코드 5-27

- 훈련과 테스트 데이터의 정확도 확인
- 훈련 데이터의 오차 그래프 확인



에포크 진행될수록 정확도 상승



에포크 진행될수록 오차가 낮아짐

#코드 5-28. 예측 이미지 출력을 위한 전처리 함수

- `tensor.clone()`: 기존 텐서의 내용을 복사한 텐서를 생성
`detach()`: 기존 텐서에서 기울기가 전파되지 않는 텐서

```
image = tensor.clone().detach().numpy()
```

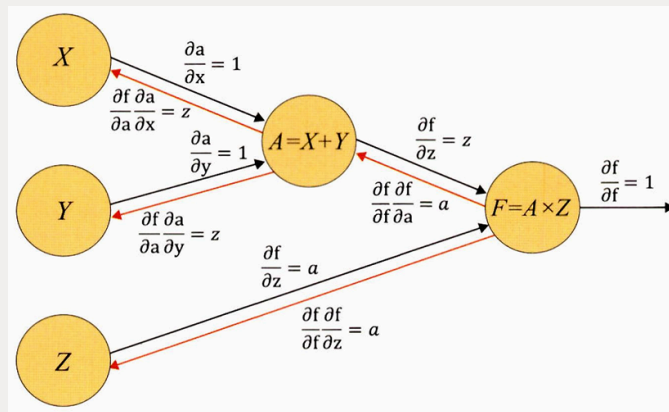
기존 텐서를 복사한 텐서를 생성하지만 기울기에 영향X

- `tensor.clone()`, `tensor.detach()`, `tensor.clone().detach()` 비교 표

구분	메모리	계산 그래프 상주 유무
<code>tensor.clone()</code>	새롭게 할당	계산 그래프에 계속 상주
<code>tensor.detach()</code>	공유해서 사용	계산 그래프에 상주하지 않음
<code>tensor.clone().detach()</code>	새롭게 할당	계산 그래프에 상주하지 않음

계산 그래프

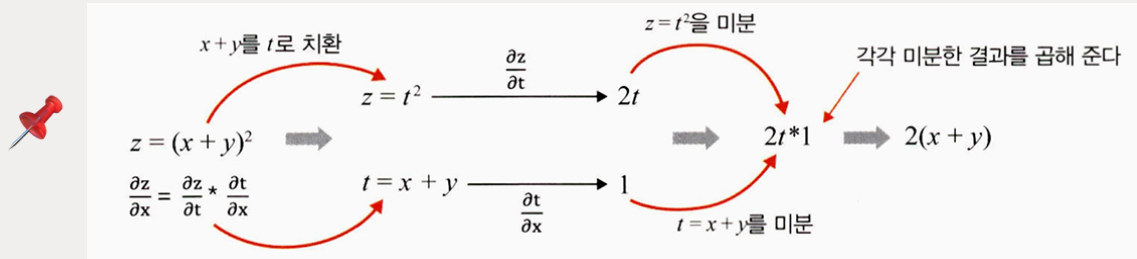
계산 과정을 그래프로 나타낸 것으로, 여러 개의 node와 edge로 구성



- 계산 그래프 사용 이유
 - 국소적 계산 가능
 - 역전파를 통한 미분 계산 편리 : 연쇄법칙 이용

연쇄 법칙

두 개 이상의 함수가 결합된 함수, 즉 합성 함수의 미분법



$z = (x+y)^2$ 를 $z=t^2$, $t=x+y$ 의 합성 함수로 표현, 각 합성 함수의 미분 결과를 곱한 결과로 나타내는 것

- `clip()`: 입력 값이 주어진 범위를 벗어날 때 입력 값을 특정 범위로 제한시키기 위해 사용

```
image = image.clip(0,1) #image데이터를 0과 1 사이 값으로 제한
```

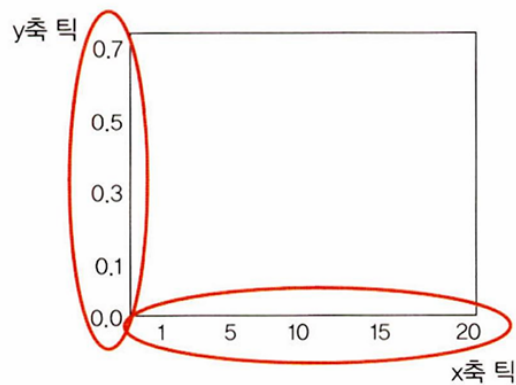
#코드 5-29. 개와 고양이 예측 결과 출력

- `add_subplot`: 한 화면에 여러 개의 이미지를 담기 위해 사용

```
ax = fig.add_subplot(2, 10, idx+1, xticks=[], yticks=[])
```

- 첫 번째 파라미터: 행의 수를 의미, 이미지를 두 줄로 출력
- 두 번째 파라미터: 열의 수를 의미, 한 줄에 열 개의 이미지 출력
- 세 번째 파라미터: 인덱스 의미, 행과 열을 기준으로 순차적으로 이미지 출력
- 네 번째 파라미터: 틱 삭제

♥ 그림 5-41 x축, y축 틱



- `classes[preds[idx].item()]` : `preds[idx].item()`값이 `classes`값 중 어떤 값인지 판별

```
ax.set_title("{}({})".format(str(classes[preds[idx].item()]),str(classes[labels[idx].item()]))),color=("green" if preds[idx]==labels[idx] else "red"))
```

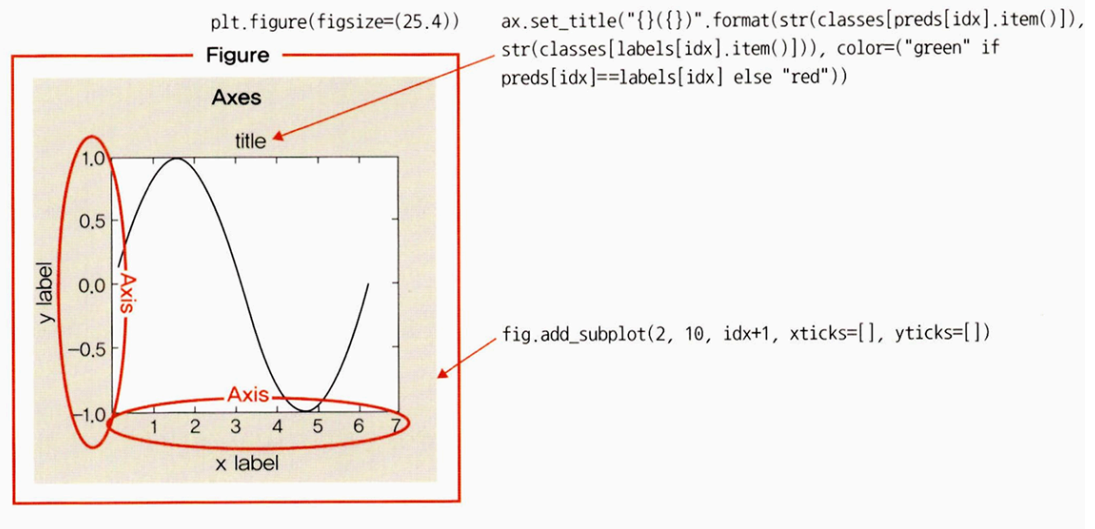
0이면 고양이, 1이면 개로 출력

- Figure 안에서 서브플롯의 위치 조정

```
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

- `left`, `bottom`, `right`, `top`으로 이미지 위치 조정
- `hspace`, `wspace`를 사용해서 서브플롯 간의 간격(너비,높이의 비율) 조정

♥ 그림 5-42 서브플롯



• 개와 고양이 예측 결과



예측이 정확하지 않음 → 훈련 데이터를 늘리고, 에포크 횟수 증가 시 더 높은 효과 기대

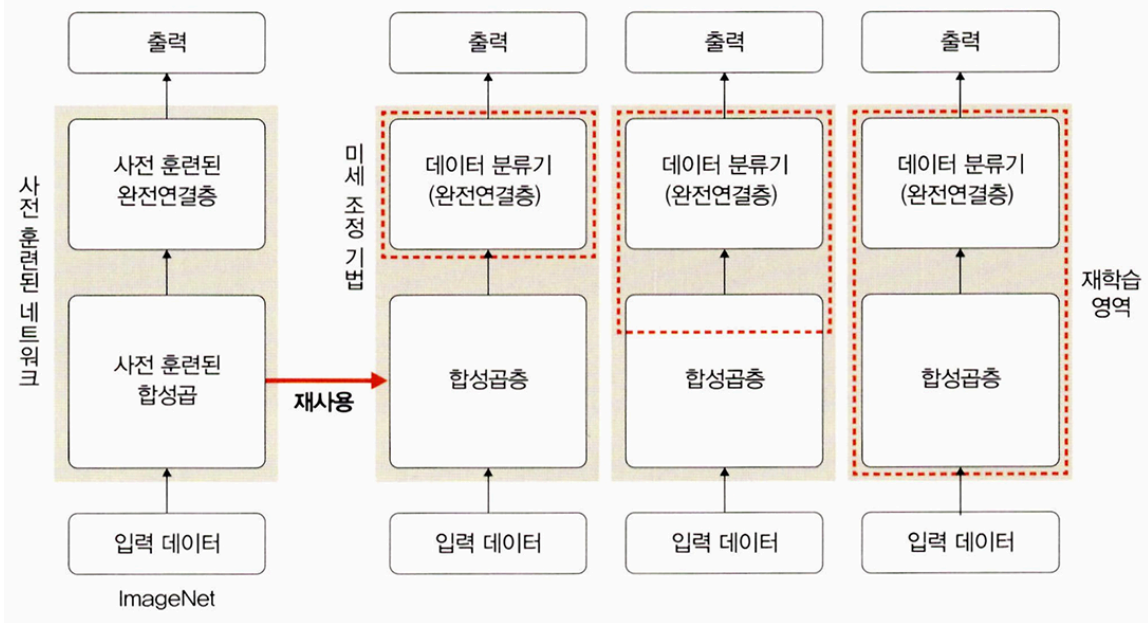
- 초록색은 개와 고양이 정확히 예측, 빨간색은 잘못된 예측

5.3.2 미세 조정 기법

사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식

- 특성 추출은 목표 특성을 잘 추출했다는 전제 하에 좋은 성능
- 특성이 잘못 추출되었다면 미세 조정 기법으로 특성을 다시 추출
- 사전 훈련된 네트워크를 미세 조정하여 분석하려는 데이터셋에 잘 맞는 모델의 파라미터를 조정하는 기법 → 많은 연산량 필요, GPU 사용 권장
- 훈련 데이터셋 크기와 사전 훈련된 모델에 따라 다른 미세 조정 기법

▼ 그림 5-44 미세 조정 기법



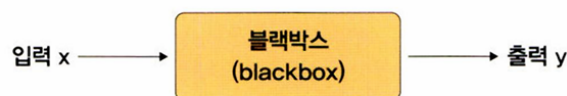
- 데이터셋이 크고, 사전 훈련된 모델과 유사성이 작은 경우: 모델 전체 재학습
- 데이터셋이 크고, 사전 훈련된 모델과 유사성이 클 경우: 합성곱층의 뒷부분(완전연결층과 가까운 부분)과 데이터 분류기 학습
- 데이터셋이 작고, 사전 훈련된 모델과 유사성이 작을 경우: 합성곱층의 일부분과 데이터 분류기 학습 but 데이터가 작아서 기법 적용을 해도 효과가 없을 가능성
- 데이터셋이 작고 사전 훈련된 모델과 유사성이 클 경우: 데이터 분류기만 학습, 데이터가 작아서 많은 계층에 미세 조정 기법 적용 시 과적합 발생 우려

5.4 설명 가능한 CNN

딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술

- CNN으로 얻은 결과는 신뢰하기 어려운데, 이를 해결하기 위해 CNN 처리 과정을 시각화해야 할 필요성이 있음

▼ 그림 5-45 CNN의 블랙박스



- CNN의 시각화 방법에는 필터에 대한 시각화, 특성 맵에 대한 시각화가 있음

5.4.1 특성 맵 시각화

특성 맵(feature map)

입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과로, 특정 이미지에 대한 특성 맵을 시각화한다는 의미는 특성 맵에서 입력 특성을 감지하는 방법을 이해하도록 돕는 것이다

- PIL(Python Image Library) 설치 : 이미지 분석 및 처리를 돕는 라이브러리
 - 다양한 이미지 파일 형식 지원
 - 강력한 이미지 처리와 그래픽 기능을 제공

```
!pip install pillow
```

#코드 5-30. 필요한 라이브러리 호출

#코드 5-31. 설명 가능한 네트워크 생성

- 13개의 합성곱층과 두 개의 완전연결층으로 구성된 네트워크 생성
- 합성곱층과 완전연결층은 ReLU 활성화 함수 사용
- `log_softmax()`: 신경망 말단의 결괏값들을 확률 개념으로 해석하기 위하여 사용

```
return F.log_softmax(x)
```

💡 로그 소프트 맥스 사용 이유는?

소프트맥스는 기울기 소멸 문제에 취약하기 때문이다

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$

$$\text{logsoftmax} = \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right)$$

$$= x_i - \log\left(\sum_{j=1} e^{x_j}\right)$$

소프트맥스와 로그 소프트맥스에 대한 수식

#코드 5-32. 모델 객체화

- 생성된 모델을 `model = XAI()` 로 객체화한 후 장치에 할당

#코드 5-33. 특성 맵을 확인하기 위한 클래스 정의

- 특성 맵: 합성곱층을 입력 이미지와 필터를 연산하여 얻은 결과
- 합성곱층에서 입력과 출력을 알 수 있다면 특성 맵에 대한 값들 확인 가능
- hook기능을 사용하여 각 계층의 활성화 함수 및 기울기 값 확인

```
self.hook = model[layer_num].register_forward_hook(self.hook_fn)
```

- `register_forward_hook`: 순전파 중에 각 네트워크 모듈의 입력 및 출력을 가져오는 것
- hook를 이용하면 중간 결괏값 확인 가능

#코드 5-34. 이미지 호출

- `cv2.resize`: 이미지 크기 변경 시 사용

```
img = cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR)
```

- 첫 번째 파라미터 `img` : 변경할 이미지 파일
- 두 번째 파라미터 `(100, 100)`: 변경될 이미지 크기를 (너비, 높이)로 지정
- `interpolation`: 보간법

- 보간법: 이미지상에 존재하는 픽셀 데이터들에 대해 근사 함수를 적용해서 새로운 픽셀 값을 구하는 것
- `unsqueeze()`: 1차원 데이터를 생성하는 함수

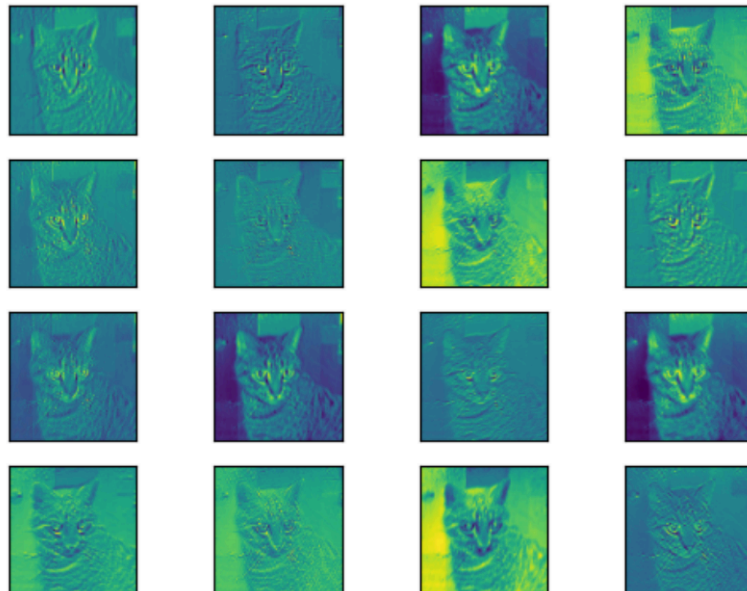
```
img = ToTensor()(img).unsqueeze(0)
```

이미지 데이터를 텐서로 변환하고, 그 변환된 데이터를 1차원으로 변경

#코드 5-35. (0): Conv2d 특성 맵 확인

#코드 5-36. 특성 맵 확인

- 출력 결과

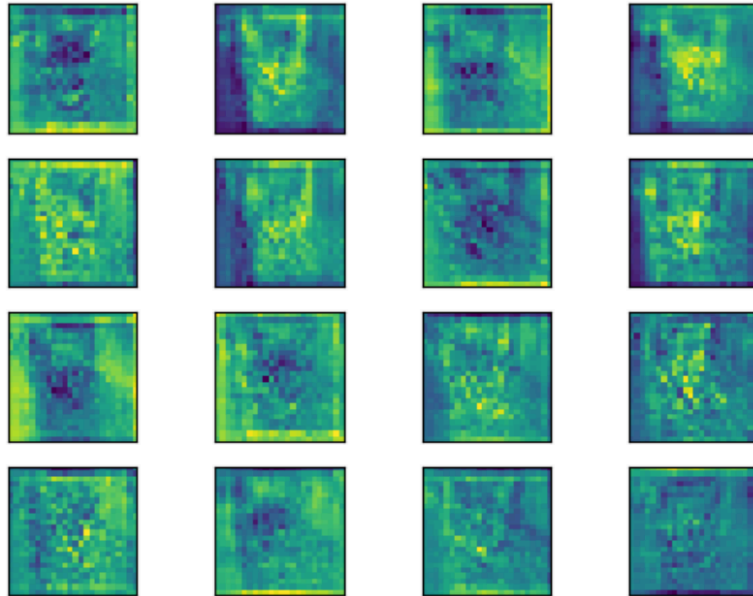


첫 번째 계층에서 특성 맵

입력층과 가까운 계층으로, 입력 이미지의 형태가 많이 유지

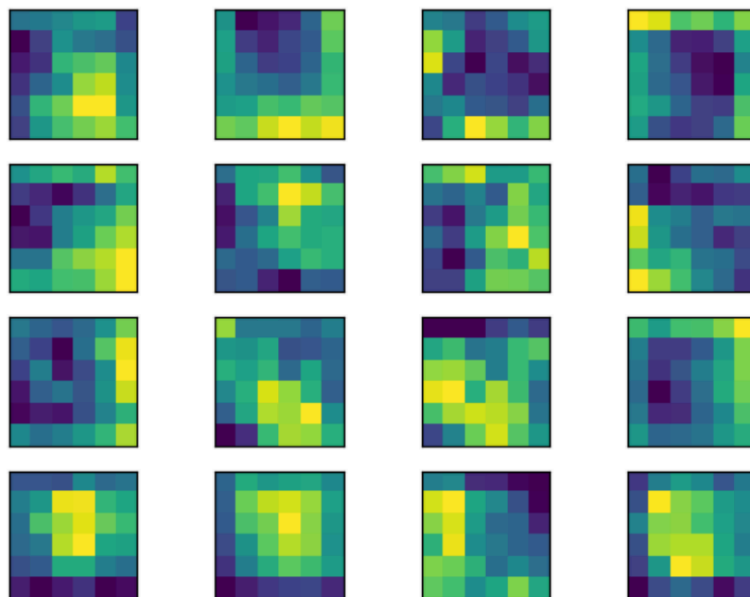
#코드 5-37, 5-38. 20번째 계층에 대한 특성 맵, 특성 맵 확인

- 출력 결과



#코드 5-39, 5-40. 40번째 계층에 대한 특성 맵, 특성 맵 확인

- 출력결과



원래 입력 이미지에 대한 형태를 찾아볼 수 없음

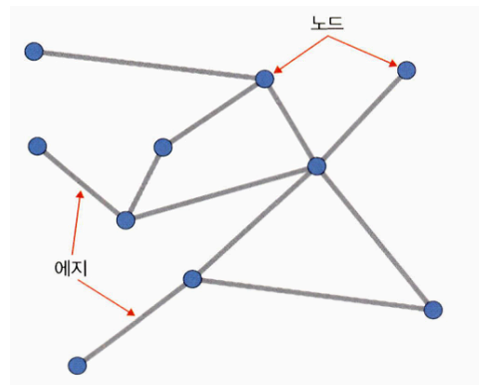
⇒ 출력층에 가까울수록 원래 형태는 찾아볼 수 없고, 이미지 특징만 전달

5.5 그래프 합성곱 네트워크

5.5.1 그래프란

방향성이 있거나 없는 에지로 연결된 노드의 집합

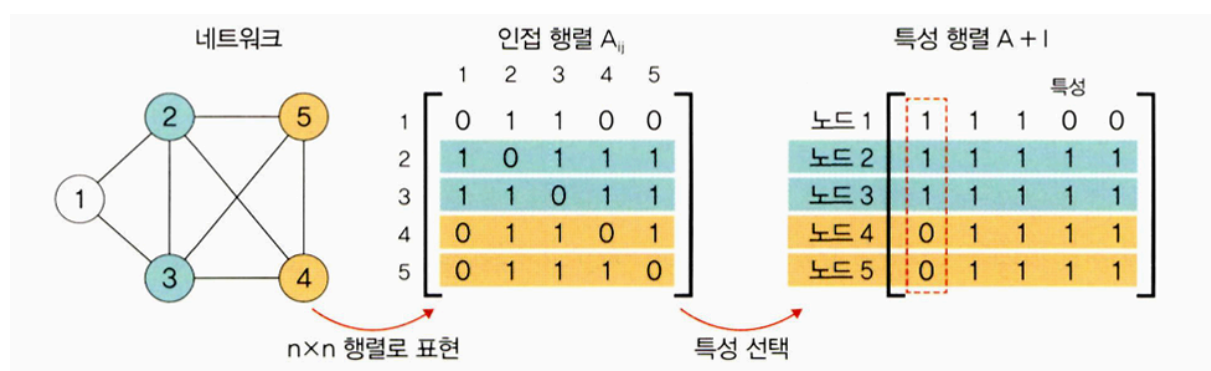
- 그래프의 형태



- 그래프의 구성 요소
 - node(vertex): 파란색 원
 - edge: 두 노드를 연결한 선
- 노드는 원소, 에지는 결합 방법(single, double, triple, aromatic 등)을 의미

5.5.2 그래프 신경망(Graph Neural Network, GNN)

그래프 구조에서 사용하는 신경망



1단계. 인접 행렬(adjacency matrix)

- 그림 왼쪽과 같이 네트워크가 있을 때 노드 n 개를 $n \times n$ 행렬로 표현
- 생성된 인접 행렬 내의 값은 'A_{ij}는 i와 j의 관련성 여부'를 만족하는 값으로 채워줌

- 컴퓨터가 이해하기 쉽게 그래프로 표현하는 과정

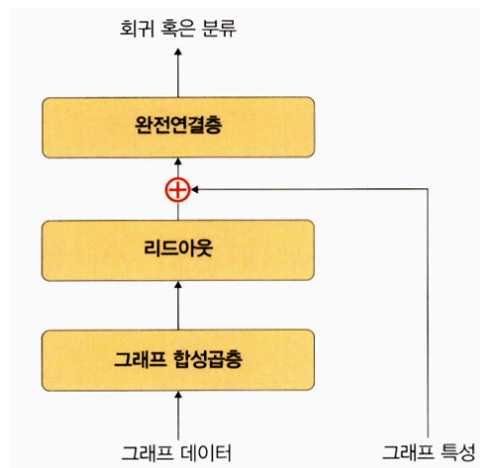
2단계. 특성 행렬(feature matrix)

- 인접 행렬만으로 특성 파악이 어려워서 단위 행렬 적용
- 각 입력 데이터에서 이용할 특성 선택
- 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미
- 특성 행렬 과정을 거쳐 그래프 특성이 추출

5.5.3 그래프 합성곱 네트워크(Graph Convolutional Network, GCN)

이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘

- 그래프 합성곱 네트워크 구조



- readout: 특성 행렬을 하나의 벡터로 변환하는 함수
 - 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터 생성
- **graph convolutional layer**: 행렬 형태의 데이터로 변환되어 딥러닝 알고리즘 적용 가능
- GCN의 활용
 - SNS에서 관계 네트워크, 학술 연구에서 인용 네트워크, 3D Mesh